

Laboratório 2: Processamento de Bases de Dados em Lote

Isabela Diniz Cia

Processamento de Bases de Dados em Lote

Tarefa

- Lendo 100.000 observações por vez (se você acredita que seu computador não tem memória suficiente, utilize 10.000 observações por vez), **determine o percentual de vôos por Cia. Aérea que apresentou atraso na chegada (ARRIVAL_DELAY) superior a 10 minutos.** As companhias a serem utilizadas são: **AA, DL, UA e US**. A estatística de interesse deve ser calculada para *cada um dos dias de 2015*. Para a determinação deste percentual de atrasos, apenas verbos do pacote `dplyr` e comandos de importação do pacote `readr` podem ser utilizados. Os resultados para cada Cia. Aérea devem ser apresentados em um formato de **calendário**.

Instruções

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Resposta: Para calcular esse percentual de vôos atrasados precisamos contar:

- Quantos vôos tiveram `ARRIVAL_DELAY > 10`
- Quantos vôos tiveram no total

em cada um dos dias/mês/companhia.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;

- b. Remova observações que tenham valores faltantes em campos de interesse;
- c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
- e. A função deve receber apenas dois argumentos: i. `input`: o conjunto de dados (referente ao lote em questão); ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. **É importante** observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
getStats = function(input, pos) {
  input %>%
    # a. Filtrando Cia. Aerea
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%
    # b. Removendo observacoes faltantes
    filter(!is.na(ARRIVAL_DELAY), !is.na(YEAR), !is.na(MONTH), !is.na(DAY)) %>%
    # c. Agrupando o conjunto de dados
    group_by(YEAR, DAY, MONTH, AIRLINE) %>%
    # d. Retornando stats. suficientes
    summarise(
      n_atrasados = sum(ARRIVAL_DELAY > 10, na.rm = TRUE),
      total = n(),
      .groups = 'drop'
    )
}
```

- 3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- Configure o tamanho do lote (chunk) para 100 mil registros;
- Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
- Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
mycols = cols_only(
  YEAR = 'i',
  MONTH = 'i',
  DAY = 'i',
  AIRLINE = 'c',
  ARRIVAL_DELAY = 'd'
)
```

```
in_chunked = read_csv_chunked('flights.csv',
  callback = DataFrameCallback$new(getStats),
  chunk_size = 100000,
  col_types = mycols
)

in_chunked
```

```
# A tibble: 1,478 x 6
  YEAR DAY MONTH AIRLINE n_atrasados total
  <int> <int> <int> <chr>      <int> <int>
1  2015     1     1 AA          514  1379
2  2015     1     1 DL          132  1558
3  2015     1     1 UA          367  1269
4  2015     1     1 US          201  1028
5  2015     2     1 AA          649  1508
6  2015     2     1 DL          406  2261
7  2015     2     1 UA          493  1411
8  2015     2     1 US          229  1174
9  2015     3     1 AA          767  1425
10 2015     3     1 DL          702  2021
# i 1,468 more rows
```

- Crie uma função chamada `computeStats` que:
 - Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
 - Retorne as informações em um **tibble** contendo apenas as seguintes colunas:

- i. Cia: sigla da companhia aérea;
- ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
- iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats = function(stats) {
  stats %>%
    mutate(
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = '-')) %>%
    mutate(
      Perc = n_atrasados / total) %>%
    select(
      Cia = AIRLINE,
      Data,
      Perc
    )%>%

    arrange(Data)
}

resultado_final = computeStats(in_chunked)
resultado_final
```

```
# A tibble: 1,478 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-01-02 0.430
6 DL    2015-01-02 0.180
7 UA    2015-01-02 0.349
8 US    2015-01-02 0.195
9 AA    2015-01-03 0.538
10 DL   2015-01-03 0.347
# i 1,468 more rows
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

a. Instale e carregue os pacotes `ggcal` e `ggplot2`.

- b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
- c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (`tibble` com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 - d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - i. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - ii. Retornar para o usuário a base do calendário criada acima.
- iii. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
- e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```
library(ggcal)
library(ggplot2)

pal = scale_fill_gradient(
  low = '#4575b4',
  high = '#d73027'
)

baseCalendario = function(stats, cia){
  dados_cia = stats %>%
    filter(Cia == cia)

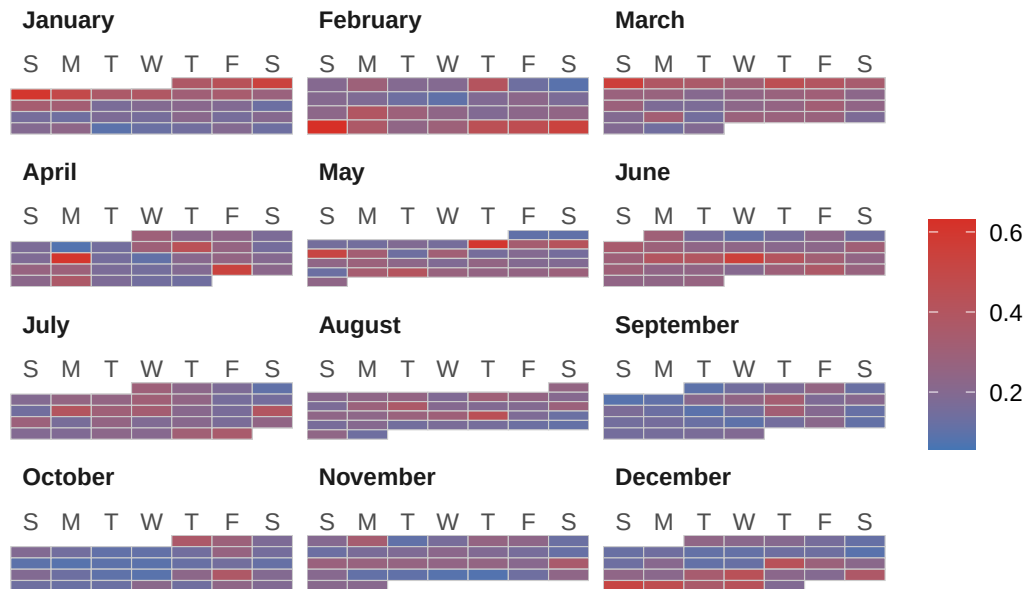
  grafico = ggcal(dados_cia$Data, dados_cia$Perc)

  return(grafico)
}

cAA = baseCalendario(resultado_final, "AA")
cDL = baseCalendario(resultado_final, "DL")
cUA = baseCalendario(resultado_final, "UA")
cUS = baseCalendario(resultado_final, "US")
```

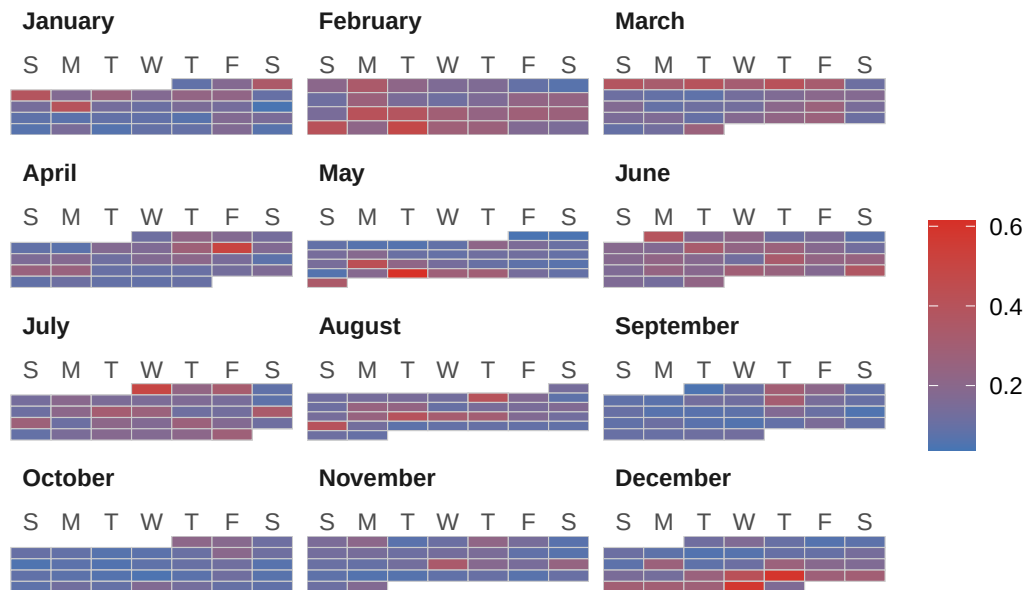
```
cAA + pal + ggtitle("American Airlines (AA)")
```

American Airlines (AA)



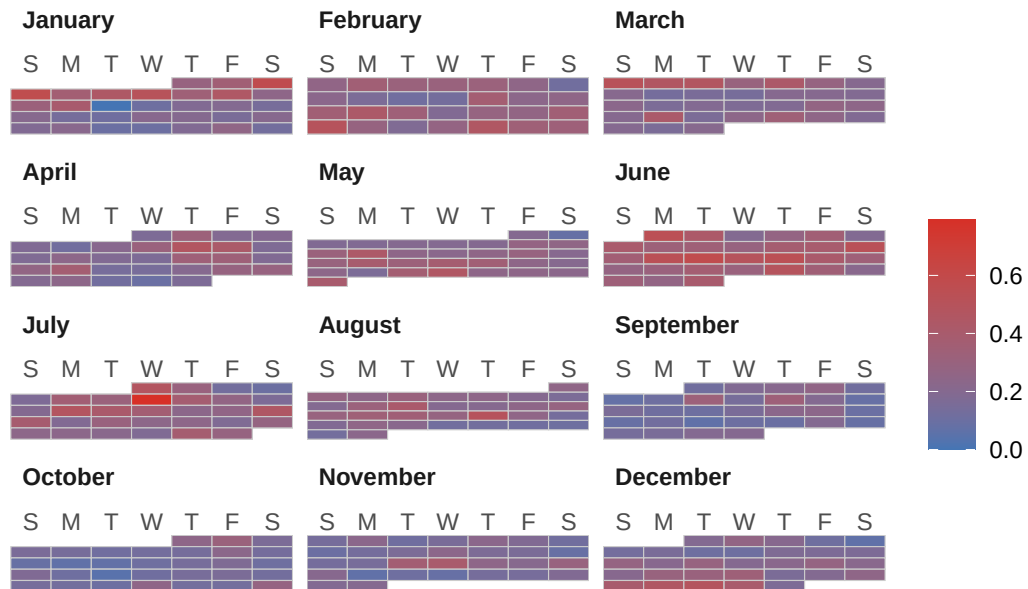
```
cDL + pal + ggtitle("Delta Airlines (DL)")
```

Delta Airlines (DL)



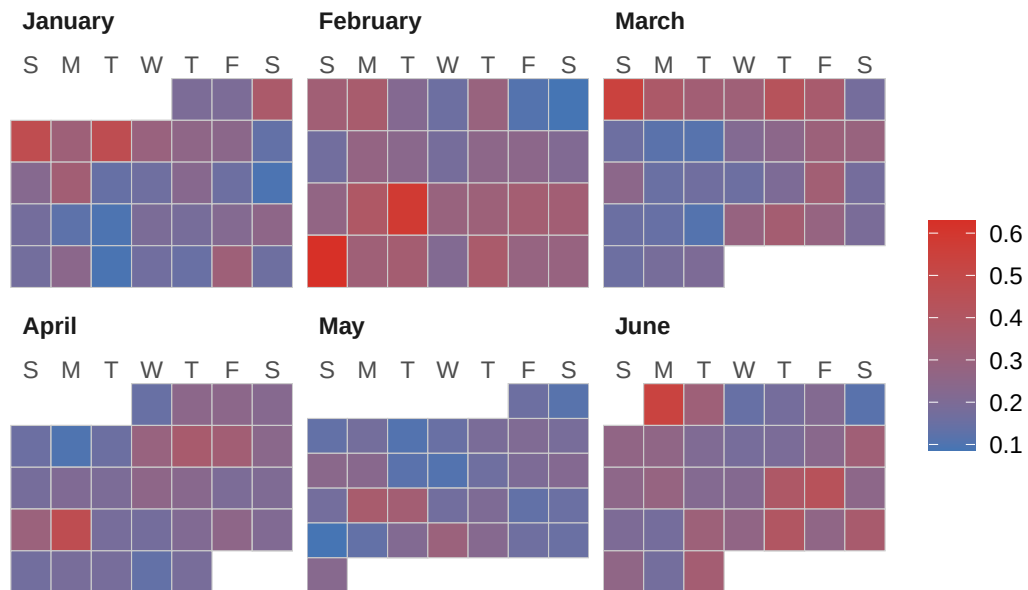
```
cUA + pal + ggtitle("United Airlines (UA)")
```

United Airlines (UA)



```
cUS + pal + ggtitle("US Airways (US)")
```

US Airways (US)



Horário de conclusão

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 18:05:12 -03"
```